DESIGN OF UART PROTOCOL USING VHDL

Submitted by :

AKRUTHI S P

CHAPTER 1:

INTRODUCTION

Universal Asynchronous Receiver & Transmitter (UART) is a kind of serial communication device that has capability to receive and transmit serial data. UART has separate transmitter and receiver modules for transmission and reception of data. UART takes parallel data as input, converts it to serial data and transmits it to receiver through channel, where serial data is again converted back to parallel data. The transmitter module controls transmission by fetching a data word in parallel format and directing the UART to transmit it in serial format. Likewise, the receiver must detect transmission, receive the data in serial format, strip of the start and stop bits, and store the data word in a parallel format. A UART is an integrated circuit which plays the most important role in serial communication. It handles the conversion between serial and parallel data. Serial communication reduces the distortion of a signal, therefore makes data transfer between two systems separated in great distance possible.

1.1 SERIAL COMMUNICATION

Serial communication is a form of communication in which only single bit is transmitted at a time. Bits are transmitted one after the other serially, either LSB first or MSB first depending upon the shifting procedure used. Serial communication is of two forms: Synchronous and Asynchronous serial communication.

1.2 UNIVERSAL ASYNCHRONOUS RECIVER AND TRANSMITTER (UART)

The UART performs serial-to-parallel conversions on data received from a peripheral device and parallel-to-serial conversions on data received from the CPU. The CPU can read the UART status at any time. The UART includes control capability and a processor interrupt system that can be tailored to minimize soft-ware management of the communications link. The UART includes a programmable baud generator capable of dividing the UART input clock by divisors producing 8 to 16 reference clocks for the internal transmitter and receiver logic.The sender does not know whether the receiver has examined the value of the bit. The sender only knows when the clock permits to start transmitting the next bit of the word. When the entire data word has been sent, the transmitter may add a parity bit that the transmitter generates. Then at least one stop bit is sent by the transmitter. When the receiver has received all of the bits in the data word, it will check for the parity bit (both sender and receiver must agree on whether a parity bit is to be used), and then the receiver looks for a stop bit. If the stop bit does not appear, receiver considers entire word to be garbled and will report a framing error to the host processor. The usual cause of a framing error is the variable speeds of the sender and receiver clocks, or the interruption of the signal. Modern modems (rated at 2400 to 19,200 bps) physically operate at or below 2400 baud, which more accurately means 2400 symbols per second. To achieve higher data rates, these modems use a technique called "constellation stuffing" to encode more bits of data into each symbol. This increases the effective bits per second even though the actual symbol rate (baud)

remains lower to fit the limited bandwidth of telephone systems. Modems operating at 28,800 bps or higher may have variable symbol rates.

1.2 PRINCIPAL OF UART

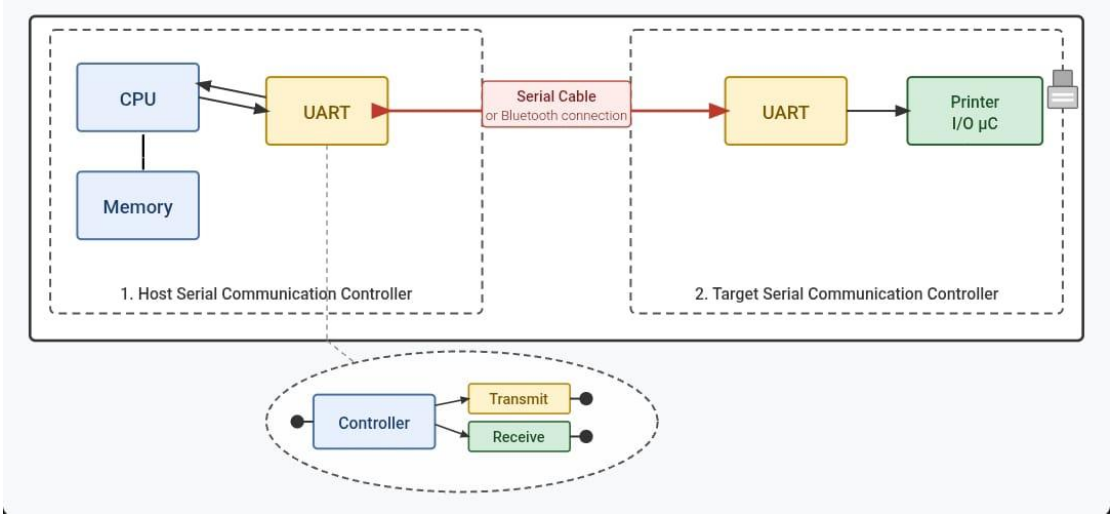


Fig1.2: Principle of UART

Set master enable; set control flags; load 8-bit data word for transmit sequence. Loads 'transmit data' register; buffer with start, stop and parity bits; set 'ready to send' flag .Reset counter; shift each of 10-bits onto serial bus; increment transfer counter .Check and clear status flags; setup for next transmit.

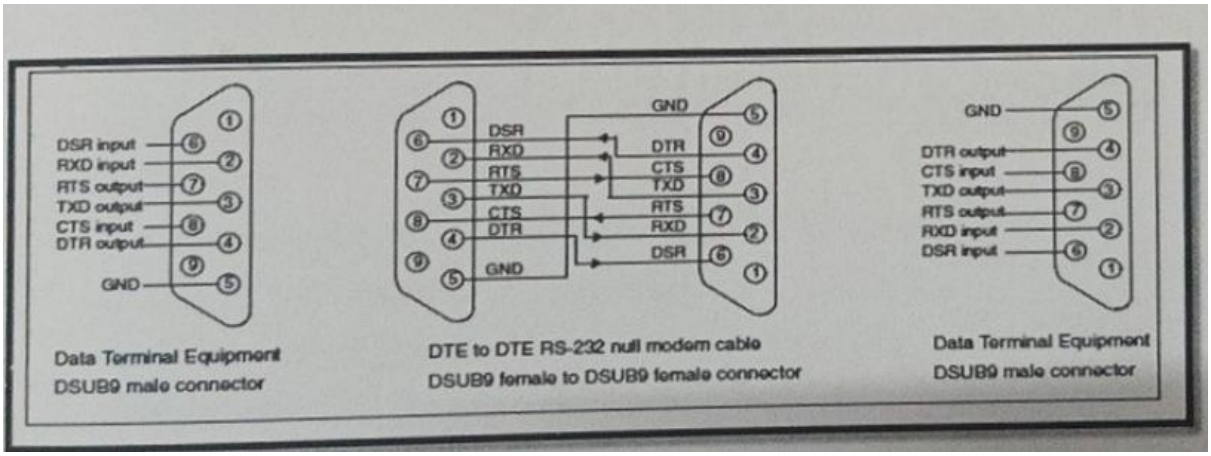


Fig1.3: DTE to DTE connection

1.3 APPLICATIONS FOR UART

Inter board communications: UART is used in many of board to board communications, where there is need for serial data transmission and conversion of data between parallel to serial and serial to parallel formats

Simple PC to Board interfacing: UART is used in most of PC to board interfacing (FPGA, CPLD etc), where serial communication with constant data rate is needed.

In Telecommunication: In telecommunication, parallel communication between DTEs or DCEs is impractical, because of need of multiple wires for parallel data transmission, which increases the cost and complexity of transmission. So serial communication is used for data transmission using UART as interface, which is effective and efficient.

CHAPTER 2:

UART ARCHITECTURE

UART architecture consists of 3 main modules,

- Transmitter
- Receiver
- Baud rate generator

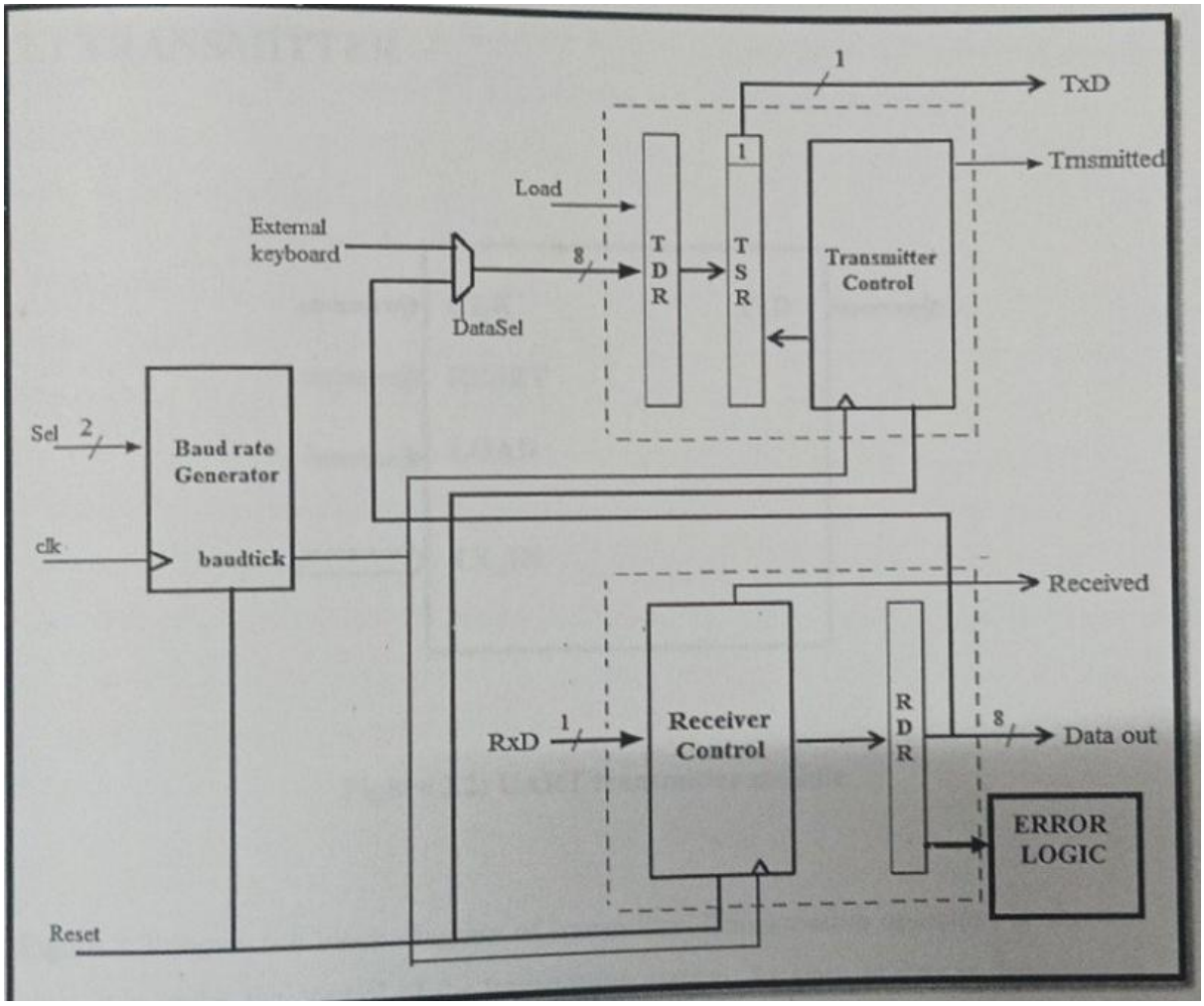


Fig 2.1: UART architecture

Transmitter acts as source of communication to every UART serial communication. Transmitter takes the parallel data as input and transmits it as framed serial data. The reference time of transmitter and receiver will be the baud clock, which is fed

as input to both of these modules. The receiver will have reference time as baud clock and error logic will make it further more efficient and effective for error free communication. Baud rate generator in the architecture presented in this report is a separate module and capable of generating variable baud rates depending upon the value of select bits.

2.1 TRANSMITTER

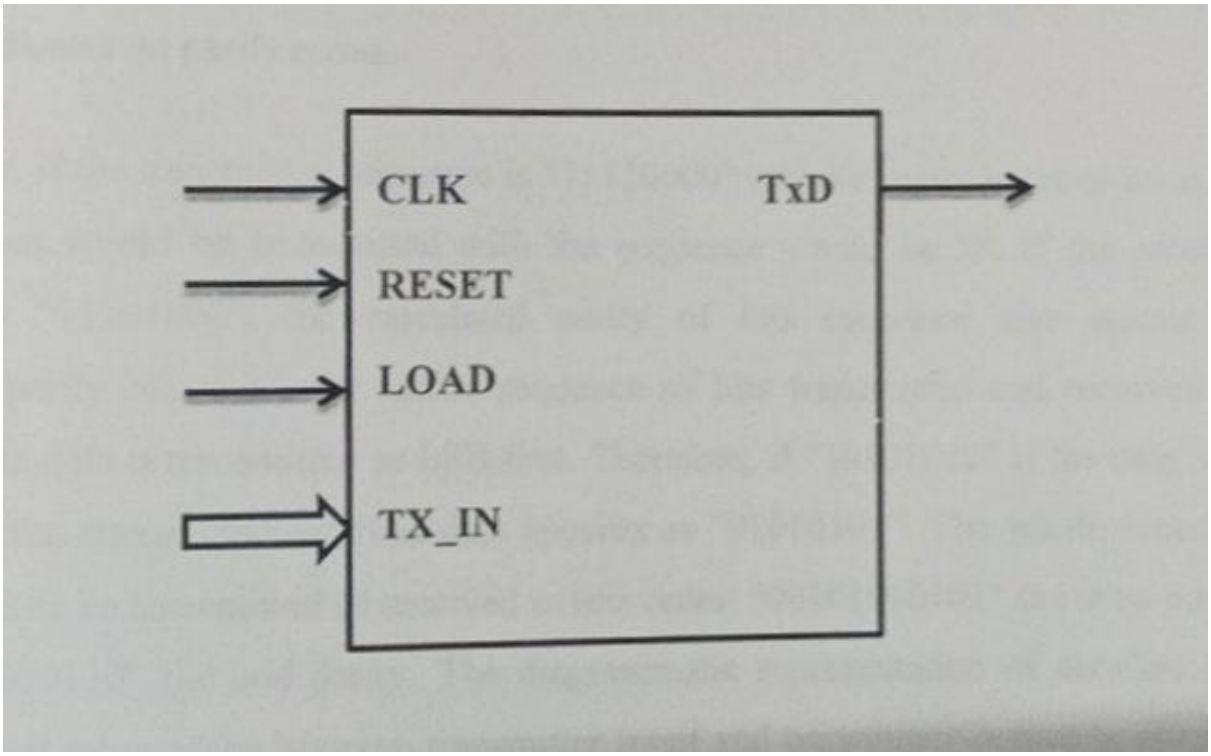


Fig 2.2: UART Transmitter module

shows the block diagram of transmitter. Transmission operation is simpler, since it is under the control of the transmitting system. As soon as data is deposited in the shift register after completion of the transmission of previous character, the UART hardware generates a start bit, shifts the required number of data bits out to the line, generates and appends the parity bit (if used), and finally appends the stop bits.

the time, practiced UART use two different DET registers for transmitted characters and received characters of separate

transmitter and receiver Transit and receive fine of the UART are held high while no option is taking place. In the transmission of a sequence, the active low indicates

to the receiving UART that a new sequence of data is on its way. This comes the receiving UART to take the next 8 bits as the transmitted data and the bit after that as the purity of these & data-bits. Lastly, a high stop bit is used to indicate the end of block. The parity can be set as even or odd and is used to indicate whether or not there has been an error in the received data bits. However, errors can still occur even if the purity hit indicates no parity errors.

For example, if the transmitted sequence is "11110000" and the parity is set as even, the parity bit that would be transmitted with the sequence would be . If the received sequence is "11101000", the calculated parity of this sequence also equals the transmitted parity bit of O, but actual sequence of bits transmitted and received are different. The data is transmitted as LSB first. Therefore, if "10101010" is the data to be transmitted, the transmitted/received data appears as "01010101". The whole sequence would therefore be transmitted or received in this order: "00101010101" for even parity, and "00101010110" for odd parity. The diagrammatic representation of detailed data processing that takes place between transmitter input and transmitter output is given in Figure 2.3

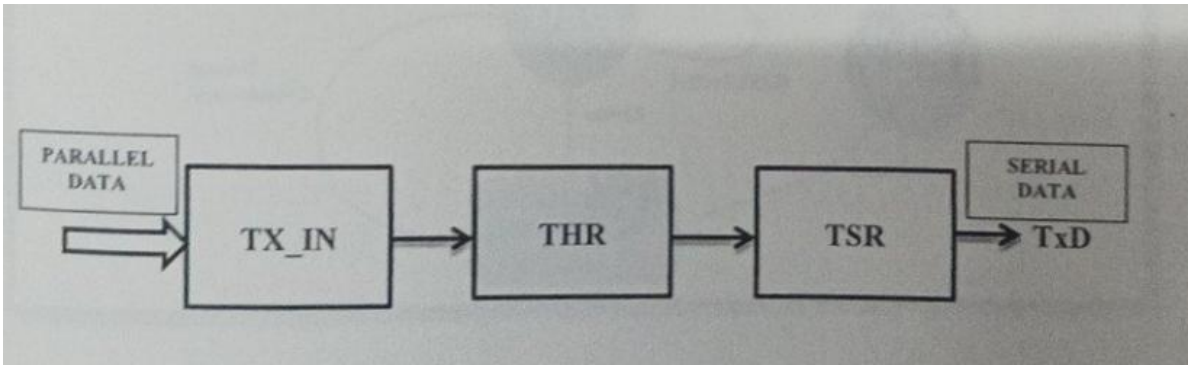


Fig 2.3: Data processing in transmitter

Transmitter receives & bits of parallel data from input pin and stores it in 8-bit register. Later data is moved to another 8-bit register, where parity of data is calculated by performing XOR operation and later data is shifted bit by bit to right. Shilling converts parallel data to serial data. As a further process parity bit and synchronizing bits (Start and stop bits) are added to serial data frame for synchronization and error free transmission. First bi First out (FIFO) register transfers UART data frame bit by bit to transmitter output pin, which is TxD in this case. Figure 2.4 shows detailed state diagram of transmitter, which includes intermediate states and condition of respective states. Data from transmitter input is loaded into THR if and only if load-1 and reset-0. If state of any one of these control input pins differ from state mentioned above, then data will not be loaded into THR. A 3 bit counter counts every bit of data received and processed and "transmitted" pin goes high when complete data frame is transmitted.

2.2 RECIEVER

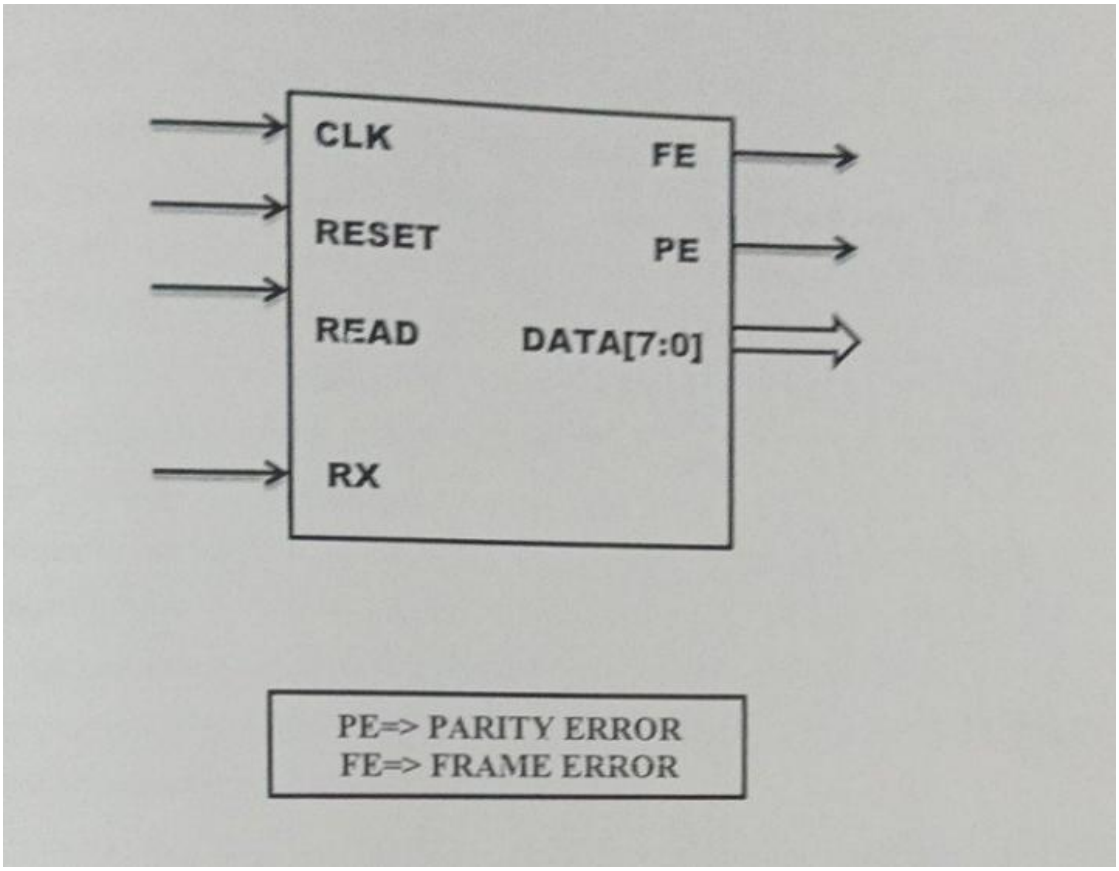


Fig 2.4: UART receiver module

Figure 2.4 shows block diagram of the receiver module. All the operations of the UART hardware are controlled by a clock signal, which runs at multiple times of the data rate. The receiver tests the state of the incoming signal on each clock pulse, looking for the beginning of the start bit. If the apparent start bit lasts at least one-half of the bit time, it is valid and signals the start of a new character. If not, the pulse is ignored. After waiting a further bit time, the state of the line is again sampled and the resulting level clocked into a shift register. After the required number of bit periods for the character length (5 to 8 bits, typically) have elapsed, the contents of the shift register is made available (in parallel fashion) to the receiving system.

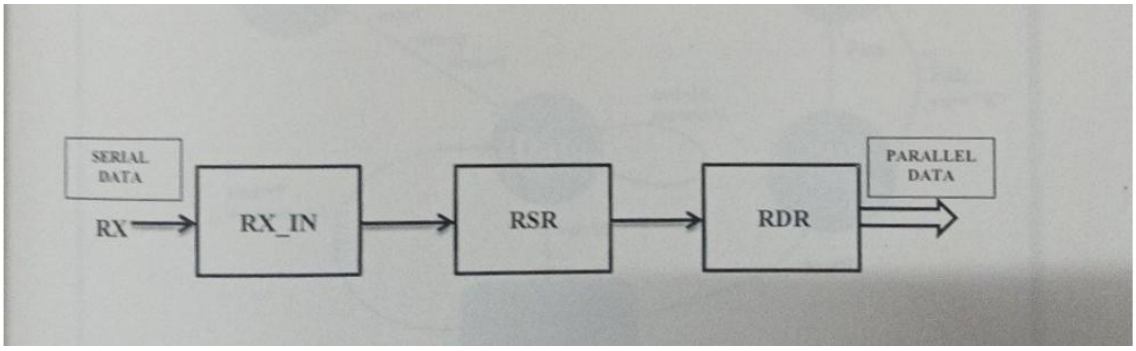


Fig 2.5: Data processing in receiver

Error detection is done in the receiver; two types of errors are detected in the implemented model. They are:

Frame error Detection: Frame error occurs when the receiver is expecting the start bit and does not find it. It is also known as a synchronization failure.

Parity error Detection: Parity error occurs when parity of transmitter does not match with parity calculated at transmitter. Parity is calculated by performing XOR operation on the data bits.

2.3 BAUD RATE GENERATOR

Baud is synonymous to symbols per second or pulses per second. It is the unit of symbol rate, also known as baud rate or modulation rate. It is the number of distinct symbol changes (signaling events) made to the transmission medium per second in a digitally modulated signal.

The Baud Rate Generator divides a base clock, using free-running counters to produce necessary timing signals like Bclk and bclkX8 for UART operations. It typically generates standard baud rates through integer division. Furthermore, a typical baud rate generator system includes counters and match detection circuits to establish and verify transmission and reception rates, with a control section to manage mismatches.

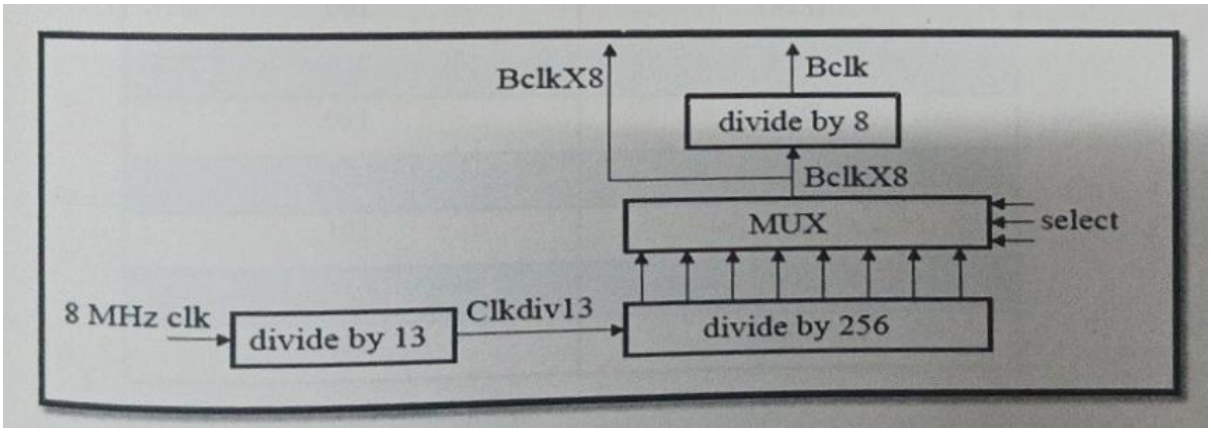
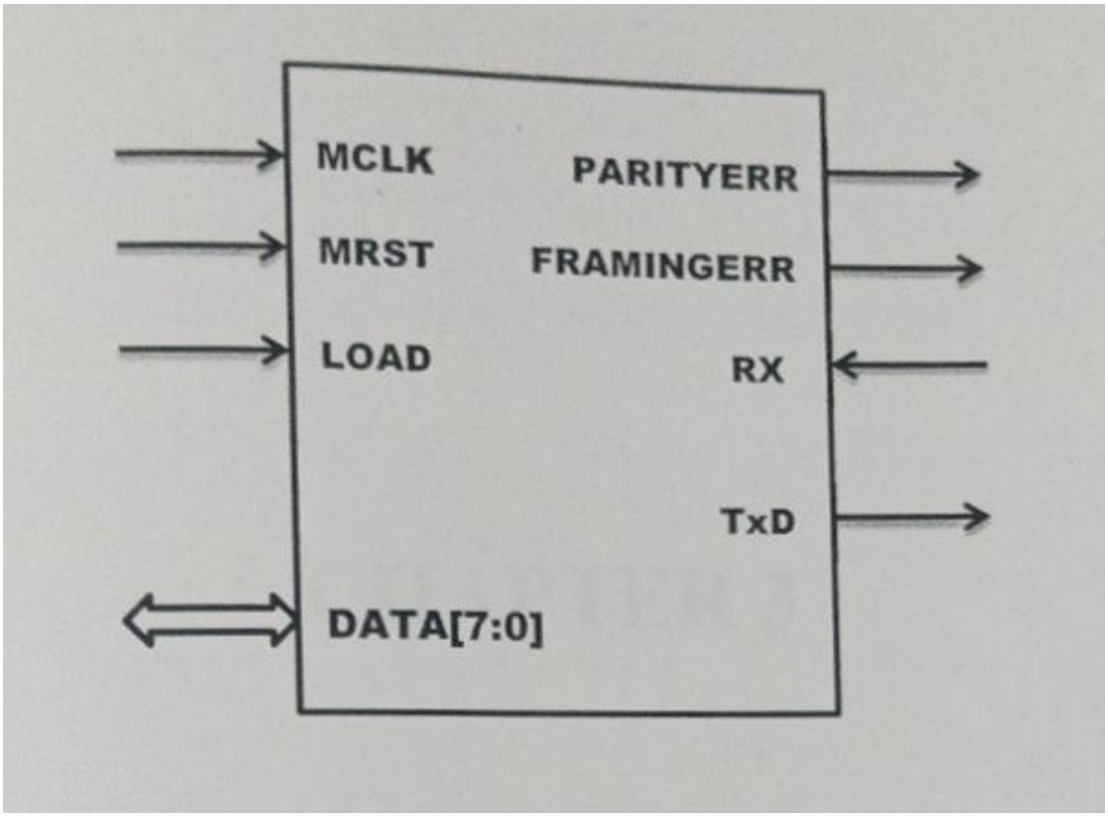


Fig 2.6: Baud rate generation process

Figure 2.6 shows the flow of process involved in baud rate generation. Three select bits are used to select any one of the baud rate from available options. Assume that 8 MHz clock is available, Initially 8 MHz system clock is divided by 13 using a counter. This counter output goes to an 8-bit binary counter. Output of the flip-flops in this counter corresponds to divide by 2, divide by 4 and so on till divide by 256. Equation (1) provides formula to calculate divisor. One of these outputs is selected by a multiplexer. MUX selects inputs from 3 bits of select input. MUX output corresponds to \(BclkX8\), which is further divided by 8 to give \(Bclk\). Equation (1) provides formula to calculate desired baud rate depending on divisor obtained. For 8 MHz clock, the possible baud frequencies generated are given in following table 2.1.

2.4 UART MODULE



UART module is the main module, where every submodule of the system is integrated and constitutes to a desired working model of UART. Figure 2.9 shows the block diagram of the complete UART module. In this implementation, fuctionality of three modules are combined in top module.

The three modules are

- Transmitter
- Receiver
- Baud rate generator

Transmitter and receiver will use baud clock as reference for communication. The complete module implemented in this project will have a transmitter, which sends data to receiver of neighbouring UART, which acts as source and receiver which will receive data from neighbouring UART, which acts as destination.

CHAPTER 3

RESULTS AND DISCUSSIONS

3.1 SIMULATION RESULT OF TRANSMITTER

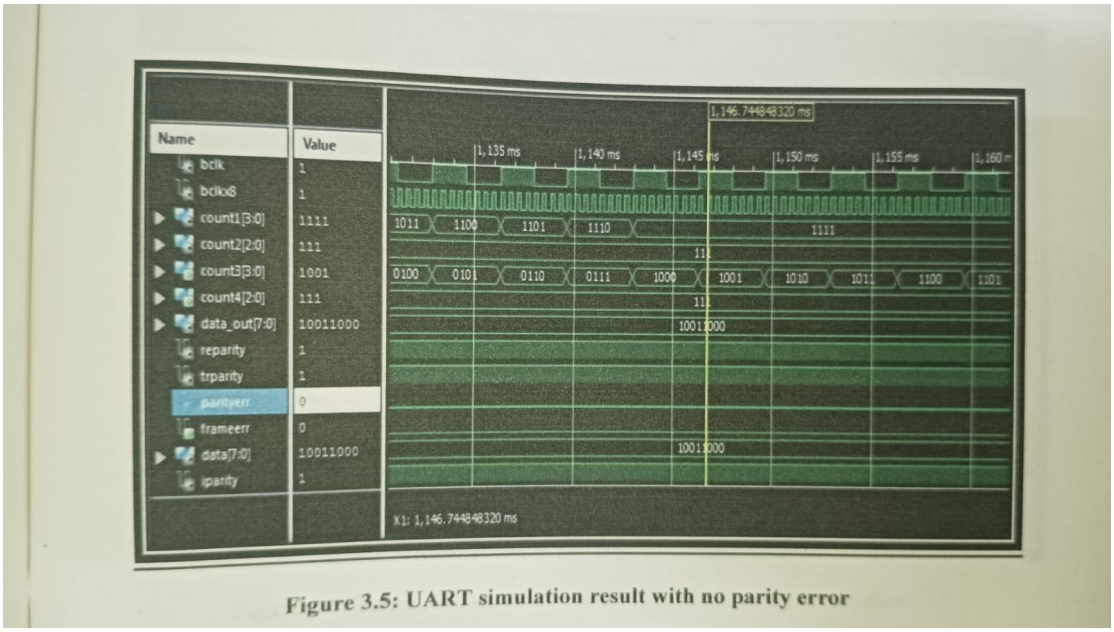


Fig 3.1 : Transmitter simulation result

Figure 3.1 shows the simulation result of transmitter module. Transmitter receives parallel input data through "din" input pin. Transmitter calculates parity and attaches parity and synchronous bits (start and stop) to data after converting parallel data to serial data bits. The resulting data frame is available at output pin "TxD". In result shown in figure 3.1, start bit of the data frame is transmitted at timing of 180mu s. After start bit, 8 data bits follows. At 330mu s parity bit is transmitted and at 350mu s stop bit is transmitted.

3.2 SIMULATION RESULT OF RECEIVER

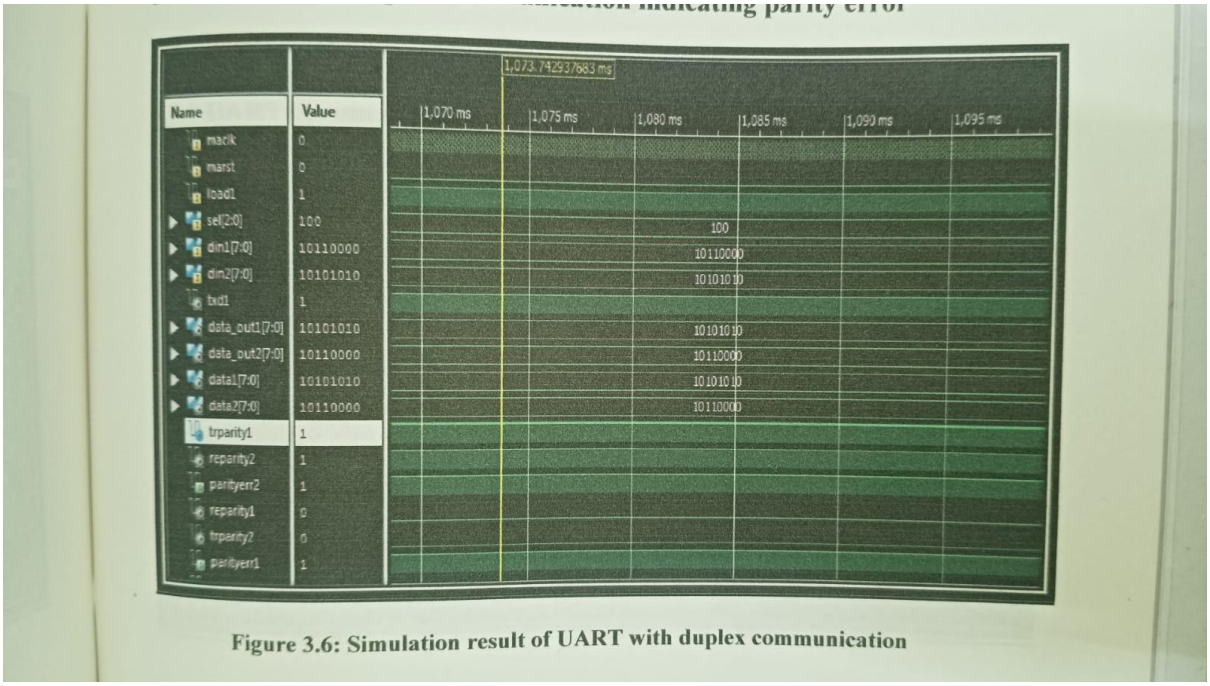


Fig 3.2 : Receiver simulation result

Figure 3.2 shows the simulation output of receiver module. Receiver receives serial input data through "rx" input pin. Receiver will detach parity and synchronous bits (start and stop) from data frame and converts serial data to parallel data bits by shifting data bits to a parallel register. The resulted parallel data is fed as output to output pin "data_out". In the result shown in figure 3.2, stot bit is detected at 39µs, followed by data bits and finally star bit of next data frame is detected at 43 µs. In case of figure 3.2, no data is fed as input to the receiver, so the received data is shown as '0' in "rx" pin.

3.3 SIMULATION RESULT OF BAUD RATE GENERATOR

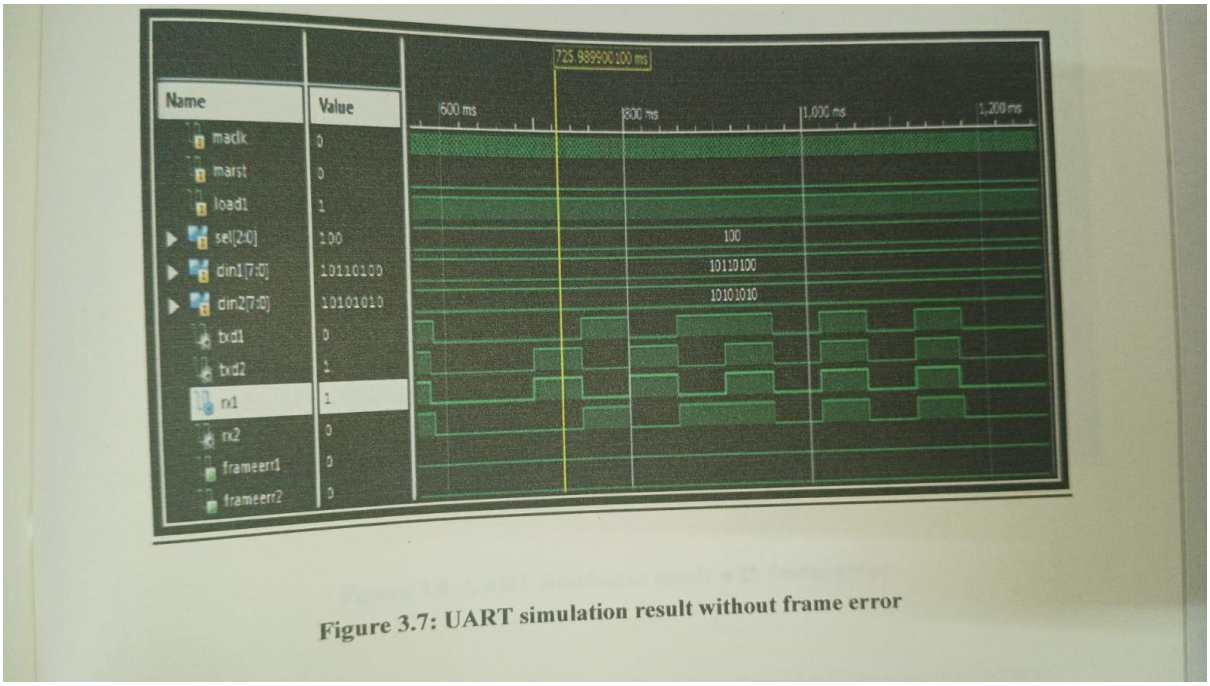


Fig 3.3 : Baud rate generator simulation result

Figure 3.3 shows simulation output of baud rate generator. Baud rate generator generates baud rate depending upon the divisor using select bits. Counter value depends on the divisor which is dependent on select input value. Depending on select input value, divisor is selected and counter value depends on divisor. So, baud rate generated depends on select input value. Here, in this simulation result shown in figure 3.3, select input has value "010". So generated baud rate is equal to 9615 baud, which is approximated as 9600 symbols per second.

3.4 SIMULATION RESULT OF UART MODULE

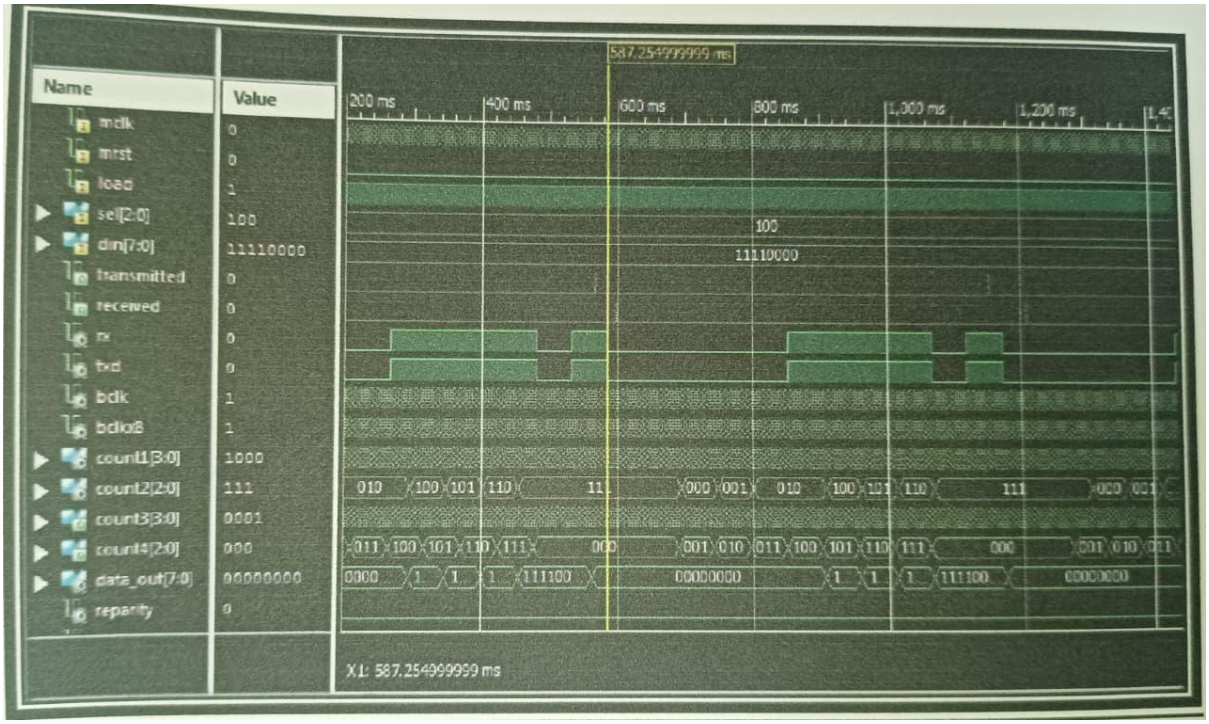


Fig 3.4 : UART simulation result

Figure 3.4 shows the simulation output of complete UART module. To demonstrate and to explain the functionality of transmitter and receiver modules, the transmitter output is fed as receiver input. Receiver takes serial data and converts it to parallel data. Parallel data is obtained as output at "data_out" pin. In simulation result shown in figure 3.4, start bit is detected at 580 ms, followed by data bits, at 1060 ms parity bit is detected and finally stop bit is detected at 1120 ms.

CHAPTER 4

CONCLUSION

Serial communication has gained importance over parallel communication for long distance communication. Asynchronous communication has an edge over synchronous communication for long distance communication, as it does not require any timing synchronization. UART is frequently used for long distance asynchronous serial communication. The architecture of UART discussed and implemented in this project supports error detection scheme and 8 different baud rate generation and selection scheme for serial transmission of data. With this architecture, different types of errors which occur during communication can be detected and indicated. Thereby disadvantages that are associated with existing UART design can be overcome. The project mainly deals with implementing of UART architecture, which is capable of supporting error free communication and possess configurable baud rate generator for variable data speed. This architecture is simple and efficient, which is suitable for most of applications.